

Data Preprocessing

1. Libraries & Data Loading

The notebook starts by importing all required libraries and loading the two CSV files into separate DataFrames.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import category_encoders as ce
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
import joblib
```

pandas and numpy handle the data. seaborn and matplotlib are used for all the plots. category_encoders provides the TargetEncoder. sklearn provides the scaler and the train/test split. joblib is used to save the trained objects to disk.

```
df_train = pd.read_csv('data/Train.csv')
df_test = pd.read_csv('data/Test.csv')

# Rename columns so both files match
df_test.rename(columns={'workclass': 'work-class', 'fnlwt': 'work-fnl', 'occupation': 'position'}, inplace=True)

# Merge into one DataFrame using a MultiIndex to track which rows came from where
df = pd.concat([df_train, df_test], keys=['train', 'test'])
```

The two files had slightly different column names, so they were renamed before merging. Using keys=['train', 'test'] creates a MultiIndex so the rows can be separated back later without losing track of which set they belong to.

2. Data Cleaning

Replace '?' with Null Values

```
df.replace(r'^\s*?\s*$', np.nan, regex=True, inplace=True)
```

The raw data stored missing values as the string '?'. A regular expression is used to catch any variation of it (with or without spaces) and replace it with a proper NaN. This makes it possible to use pandas null-handling functions on the missing data.

Encode the Target Column

```
df['salary'] = (df['salary'].str.contains('>50K')).astype(int)
```

The salary column was a string ('>50K' or '<=50K'). It was converted to a binary integer: 1 means the person earns more than \$50K and 0 means they do not. This is required for all classification models.

Remove Duplicates

```
df.duplicated().sum()    # Found 24 duplicates
df.drop_duplicates(inplace=True) df.duplicated().sum()
# Now 0
```

24 exact duplicate rows were found and removed. After removal, no duplicates remained.

Drop Redundant Columns

```
df.drop(['work-fnl', 'education'], axis=1, inplace=True)
```

Two columns were removed. The education column is just a text label that contains the same information as education-num, which is already a number. Keeping both would be redundant. The work-fnl column is a census sampling weight — it is not a real personal attribute and would only add noise to the models.

Handle Missing Values

```
# Drop rows where work-class or position are missing df.dropna(subset=['work-class',
'position'], inplace=True)

# Fill missing native-country with the most common value
df['native-country'] = df['native-country'].fillna(df['native-country'].mode()[0])
```

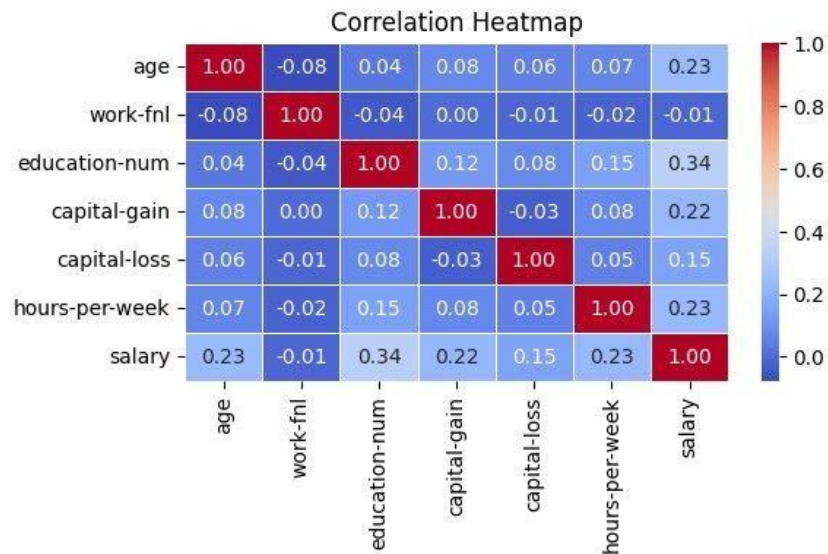
For work-class and position, rows with null values were dropped entirely because both columns describe how a person earns a living, which is directly relevant to predicting salary. Imputing random or average job types could mislead the model.

For native-country, the missing values were filled with the mode (most frequent value), which was 'United-States'. This avoids losing rows for a less critical column where the vast majority of people share the same value.

3. Exploratory Data Analysis

Correlation Heatmap

```
corr_matrix = numeric_df.corr()
sns.heatmap(corr_matrix, annot=True, cmap='coolwarm', fmt='.2f', linewidths=0.5)
plt.title('Correlation Heatmap')
```



The heatmap shows the correlation between every pair of numeric features. The values range from -1 (strong negative) to 1 (strong positive). Key observations:

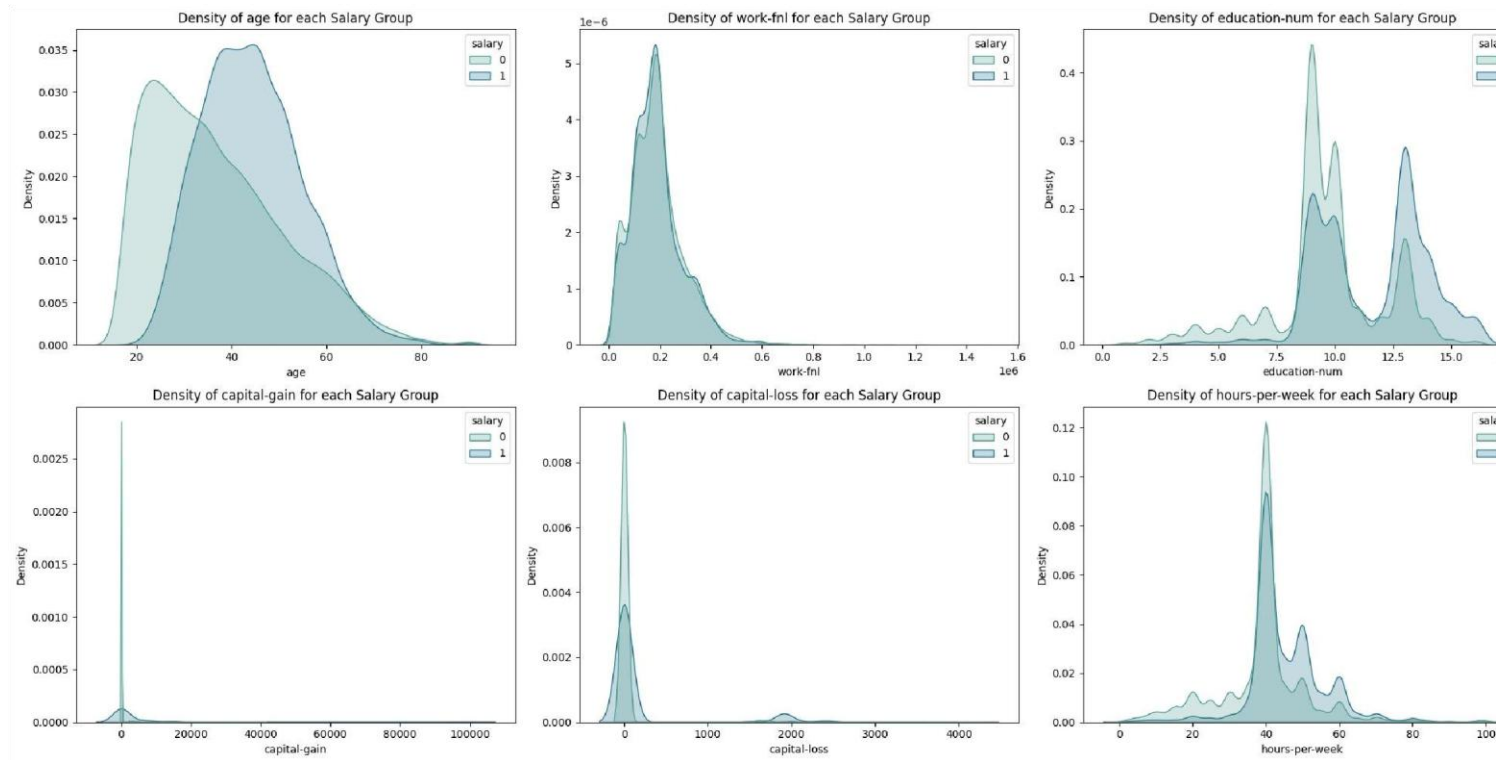
- education-num has the highest correlation with salary (0.34) — the more educated a person, the more likely they earn above \$50K.
- capital-gain also shows a noticeable link with salary (0.22), meaning investment income is a signal of higher earnings.
- age has a mild positive correlation with salary (0.23).
- work-fnl has almost zero correlation with everything, confirming it was right to remove it.
- capital-gain and capital-loss are negatively correlated with each other (-0.03), which is expected since they represent opposite financial events.

KDE Density Plots

```
for col in numeric_columns:    sns.kdeplot(data=df, x=col,
hue='salary', fill=True,      common_norm=False,
palette='crest', ax=axes[i])
```

KDE (Kernel Density Estimation) plots show the smooth distribution of each numeric feature split by salary group (0 = ≤50K shown lighter, 1 = >50K shown darker). Key findings:

- Age: The ≤50K group peaks around age 25, while the >50K group peaks around age 40-45. Higher earners tend to be older.
- education-num: The >50K group is clearly shifted to the right, concentrating between values 12-16. The ≤50K group has a larger peak at lower education levels (around 9-10).
- hours-per-week: Both groups peak at 40 hours (standard full-time). The >50K group has a slightly wider spread toward 50-60 hours.



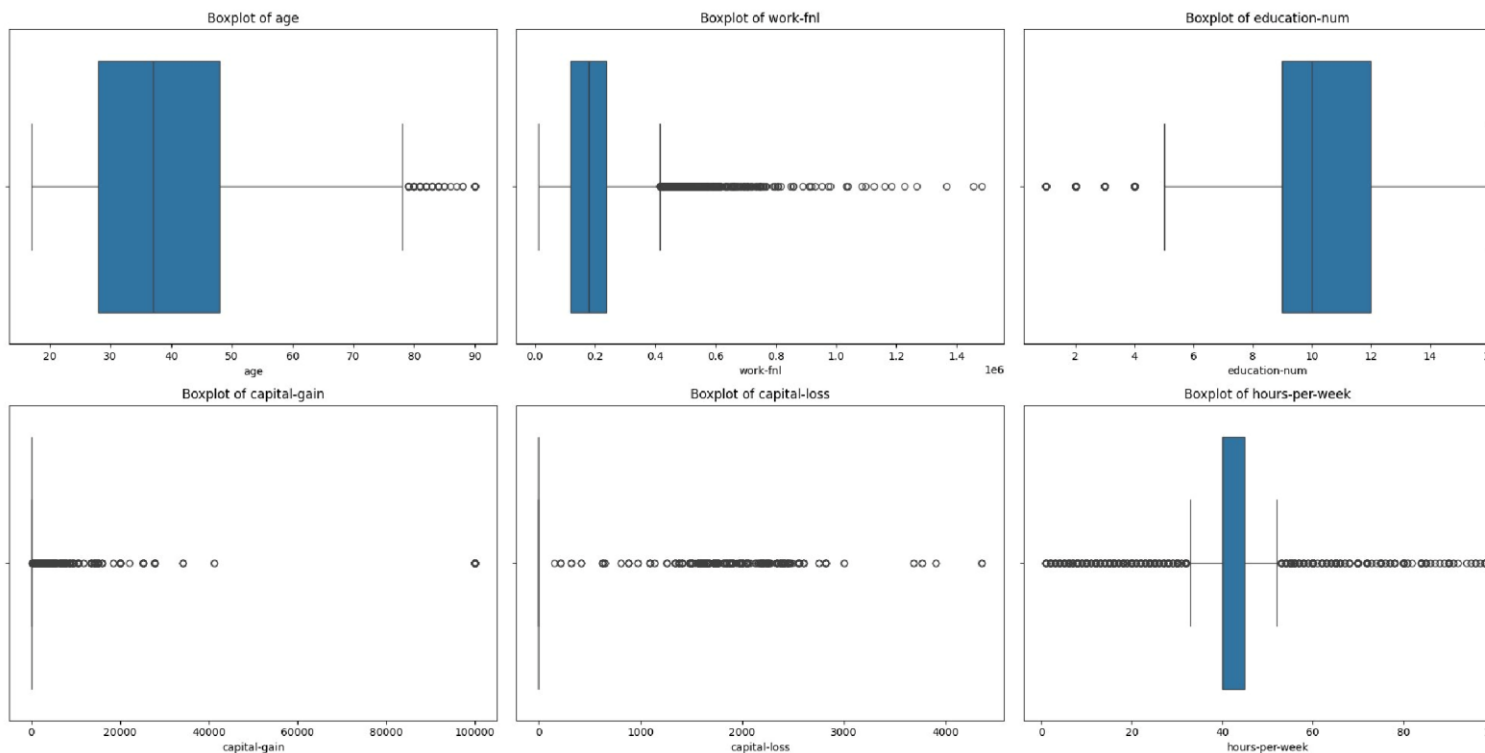
- capital-gain and capital-loss: Both are extremely right-skewed with most values at zero. However, when non-zero, they tend to appear more often in the >50K group, making them useful as predictors despite being sparse.
- work-fnl: Both groups have almost identical distributions, again confirming this feature carries no salary signal.

Box Plots

```
for col in numeric_columns:  
    sns.boxplot(data=df, x=col, fill=True, ax=axes[i])
```

Box plots show the spread and outliers for each numeric column. The box represents the middle 50% of values (IQR), the line inside is the median, and dots outside the whiskers are outliers.

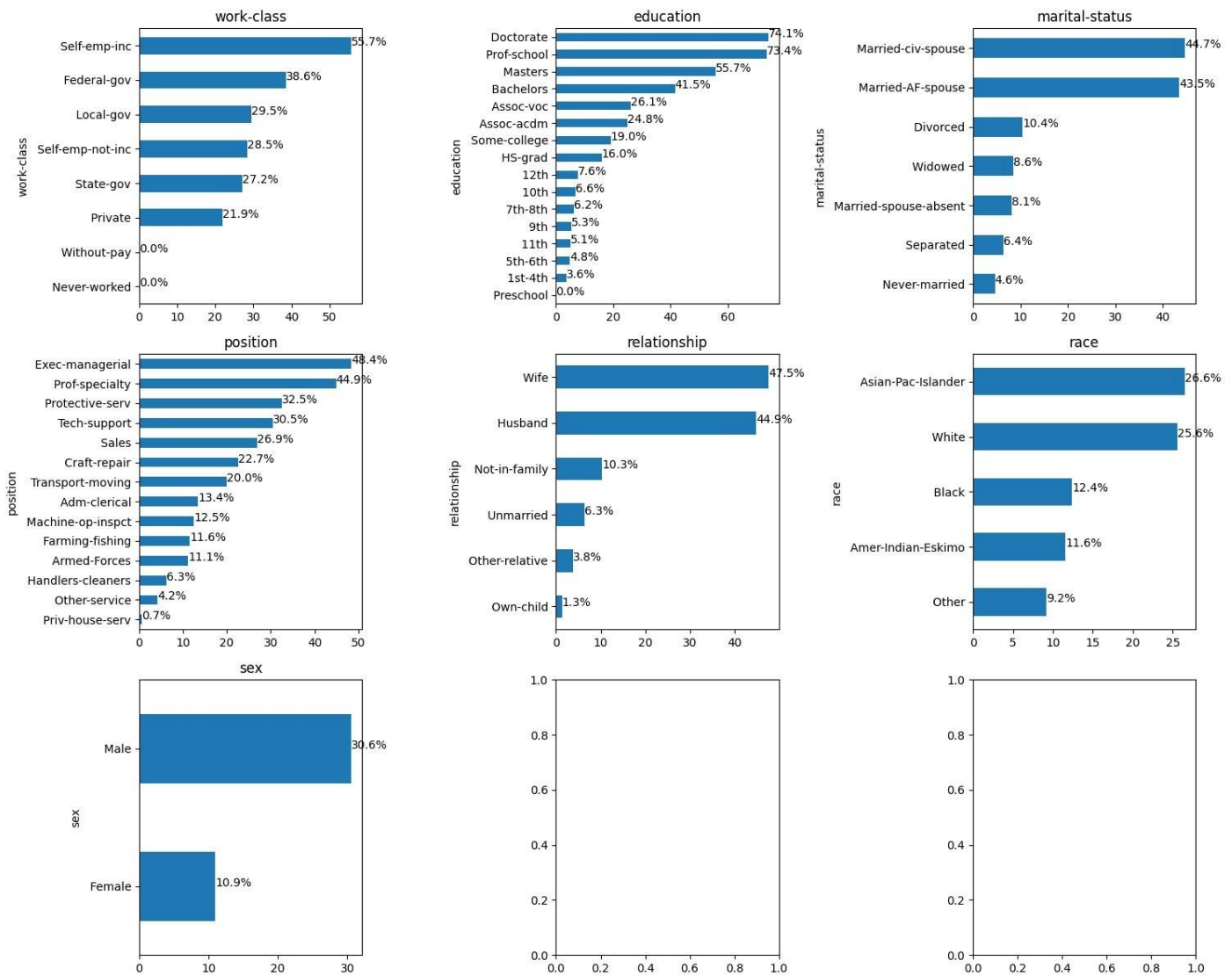
- Age: The middle 50% of people are aged roughly 28-48, with mild outliers above 80 years.
- capital-gain: Almost entirely zero with very extreme outliers reaching up to 100,000. These were kept because they represent real and meaningful financial events.
- capital-loss: Similar story — nearly all values are zero, with a cluster of outliers between 1,000-4,500. Also kept intentionally.



- hours-per-week: The box is centered at 40 hours. There are many outliers on both sides, including people who work under 20 hours and those who work over 60-80 hours per week.
- education-num: Well-distributed between 9 and 12, with a few low outliers for people with very minimal schooling.
- work-fnl: Dense at low values with many high-end outliers, reinforcing that it is a skewed and noisy variable.

Categorical Feature Bar Charts

```
for col in categorical_columns:
    feature_salary = pd.crosstab(df[col], df['salary'], normalize='index') * 100
    feature_salary[1].plot(kind='barh', ax=axes[i])
```



Each bar shows the percentage of people in that category who earn more than \$50K. This helps understand which categorical groups are most associated with high income.

- work-class: Self-employed (incorporated) workers have the highest rate at 55.7%, while government and private employees range from 22-39%. People without pay or who have never worked have 0%.
- education: Doctoral and professional school graduates top the chart at 73-74%. Bachelor's degree holders are at 41.5%. High school graduates drop to 16%, and lower education levels are well below 10%.
- marital-status: Married civilians (44.7%) and married armed-forces spouses (43.5%) have the highest rates. Divorced, separated, and never-married individuals are all below 11%.
- position: Executive-managerial (48.4%) and professional-specialty (44.9%) roles lead. Service roles like other-service (4.2%) and private house services (0.7%) are at the bottom.
- relationship: Wives (47.5%) and husbands (44.9%) — both reflecting married status — are the highest. Own-child (1.3%) and other-relative (3.8%) are the lowest.
- race: Asian-Pacific-Islander (26.6%) and White (25.6%) show the highest rates. This reflects socioeconomic patterns in the dataset.
- sex: Males earn above \$50K at a rate of 30.6% vs 10.9% for females, reflecting a significant gender income gap in the data.

4. Feature Engineering

Three new interaction features were created to capture patterns that individual columns cannot express alone. Each was validated by splitting its values into 5 equal-sized bins and measuring the average salary rate in each bin.

```
# Feature 1: Education squared times work hours
df['edu-workHours'] = df['education-num']**2 * df['hours-per-week']

# Feature 2: Education squared times age
df['edu-age'] = df['education-num']**2 * df['age']

# Feature 3: Work hours times age df['workHours-age']
= df['hours-per-week'] * df['age']

# Validation example - check if bins rise with salary rate
df['bins'] = pd.qcut(df['edu-workHours'], q=5)
print(df.groupby('bins')['salary'].mean())
```

All three features showed a clear upward pattern: as the feature value increased from bin 1 to bin 5, the percentage of high earners consistently rose. For example, edu-workHours went from around 3% in the lowest bin to over 40% in the highest. This confirms the features carry real predictive signal.

5. Encoding & Normalization

```
# Binary: Male = 1, Female = 0
df['sex'] = (df['sex'].str.contains('Male')).astype(int)

# One-Hot: columns with no natural order and few categories column_oneHot =
['marital-status', 'relationship', 'race'] df = pd.get_dummies(df,
prefix=column_oneHot, columns=column_oneHot, dtype=int)

# Target encoding: columns with many categories
column_target = ['work-class', 'position', 'native-country']
target_encoder = ce.TargetEncoder(cols=column_target, smoothing=70)
X_train = target_encoder.fit_transform(X_train, y_train)
X_test = target_encoder.transform(X_test)

# Normalize with Z-score standardization scaler
= StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_val_scaled = scaler.transform(X_val)
X_test_scaled = scaler.transform(X_test)
```

Three encoding strategies were used based on each column's properties. Sex was binary. Marital-status, relationship, and race used one-hot encoding because they have no natural order. Work-class, position, and native-country used target encoding because they have too many categories for one-hot to be practical. A smoothing factor of 70 was applied to prevent rare categories from having too much influence.

After encoding, all features were normalized using Z-score standardization. The scaler was fit only on the training data and then applied to validation and test — this prevents any information from the test set from leaking into the training process.